Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к практическому занятию №1
на тему:
«ТЕХНОЛОГИИ MMX, SIMD»
БГУИР 6-05-0612-02 113

Выполнил студент группы 453503
ХАЛАМОВ Николай Андреевич

_____
(дата, подпись студента)

Проверил ассистент каф. Информатики
МАКСИМЧИКОВ АРТЁМ ВЯЧЕСЛАВОВИЧ

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Обработать массивы из 8 элементов по следующему выражению $F[i] = A[i] * \big((B[i]) + C[i]\big) - D[i], i = 1\ldots8$. Используются следующие массивы: A, B и C – 8 разрядные целые знаковые числа (_int8); D – 16 разрядные целые знаковые числа (_int16). Полученный результат отобразить на форме с использованием соответствующих элементов. При распаковке знаковых чисел совместно с командами распаковки использовать команды сравнения (сравнивать с нулём перед распаковкой).

Цель работы – изучить технологию MMX, модель SIMD и систему команд расширения, а также получить практические навыки параллельной обработки данных с использованием ассемблерных вставок на языке C++.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для реализации поставленной задачи была использована технология *MMX* (*Multimedia Extensions*), которая представляет собой *SIMD*-архитектуру (*Single Instruction, Multiple Data*), которая позволяет выполнять инструкции над несколькими данными одновременно.

Поскольку исходные массивы A, B и C состоят из 8-битных знаковых элементов (*int8_t*), а массив D и результирующий массив F содержат 16-битные элементы (*int16_t*), перед выполнением арифметических операций над ними необходимо произвести расширение типов данных, то есть распаковку знаковых байтов в знаковые слова. Для корректной распаковки знаковых чисел предварительно формируется маска знака. Для этого используется команда сравнения с нулем.

Кроме функции, которая обрабатывает элементы массива с применением технологии *MMX*, была реализована функция, которая производит те же самые операции, но посредством обычного цикла. Это необходимо для проверки корректности вычислений.

Программа написана на языке *C++* с ассемблерными вставками и представлена в листинге 1.

Листинг 1 – Функция обработки массивов с использованием технологии MMX

```
__declspec(align(8)) int8_t A[8] = { 1, -2, 3, -4, 5, -6, 7, -8 };
__declspec(align(8)) int8_t B[8] = { 2, 3, -4, 5, -6, 7, -8, 9 };
__declspec(align(8)) int8_t C[8] = { 1, -1, 2, -2, 3, -3, 4, -4 };
__declspec(align(8)) int16_t D[8] = { 10, -20, 30, -40, 50, -60, 70, -80
};

__declspec(align(8)) int16_t F_mmx[8] = { 0 };

__declspec(align(8)) int16_t F_cpp[8] = { 0 };

__declspec(align(8)) int16_t Aw[8];
__declspec(align(8)) int16_t Bw[8];
__declspec(align(8)) int16_t Cw[8];
__declspec(align(8)) int16_t Sw[8];

void CalcMMX()
{
    __asm
    {
        lea eax, A
        movq mm0, qword ptr[eax]

        pxor mm1, mm1
        pcmpgtb mm1, mm0

        movq mm2, mm0
        punpcklbw mm2, mm1
```

```
lea eax, Aw
movq qword ptr[eax], mm2

movq mm2, mm0
punpckhbw mm2, mm1

lea eax, Aw
movq qword ptr[eax + 8], mm2

lea eax, B
movq mm0, qword ptr[eax]

pxor mm1, mm1
pcmpgtb mm1, mm0

movq mm2, mm0
punpcklbw mm2, mm1

lea eax, Bw
movq qword ptr[eax], mm2

movq mm2, mm0
punpckhbw mm2, mm1

lea eax, Bw
movq qword ptr[eax + 8], mm2

lea eax, C
movq mm0, qword ptr[eax]

pxor mm1, mm1
pcmpgtb mm1, mm0

movq mm2, mm0
punpcklbw mm2, mm1

lea eax, Cw
movq qword ptr[eax], mm2
movq mm2, mm0
punpckhbw mm2, mm1
lea eax, Cw
movq qword ptr[eax + 8], mm2
lea eax, Bw
movq mm0, qword ptr[eax]
lea eax, Cw
movq mm1, qword ptr[eax]

paddsw mm0, mm1

lea eax, Sw
movq qword ptr[eax], mm0

lea eax, Aw
movq mm1, qword ptr[eax]

pmullw mm1, mm0

lea eax, D
movq mm2, qword ptr[eax]

psubsw mm1, mm2
```

```
            lea eax, F_mmx
            movq qword ptr[eax], mm1


            lea eax, Bw
            movq mm0, qword ptr[eax + 8]
            lea eax, Cw
            movq mm1, qword ptr[eax + 8]
            paddsw mm0, mm1
            lea eax, Sw
            movq qword ptr[eax + 8], mm0
            lea eax, Aw
            movq mm1, qword ptr[eax + 8]
            pmullw mm1, mm0

            lea eax, D
            movq mm2, qword ptr[eax + 8]

            psubsw mm1, mm2

            lea eax, F_mmx
            movq qword ptr[eax + 8], mm1

            emms
        }
    }
```

В данной программе замерялось время работы нескольких итераций двух функций для обработки массива данных с применением технологии *MMX* и без, а также сравнивались полученные результаты вычислений. Результат работы программы представлен на рисунке 1.

```
mmx time: 30546 mks
c++ time: 40189 mks

 i | F_mmx | F_cpp
---+-------+-------
 1 |    -7 |    -7
 2 |    16 |    16
 3 |   -36 |   -36
 4 |    28 |    28
 5 |   -65 |   -65
 6 |    36 |    36
 7 |   -98 |   -98
 8 |    40 |    40
```

Рисунок 1 – Результат работы программы

Как видно из результатов выполнения программы, значения результирующего массива, полученные с использованием технологии MMX, полностью совпадают со значениями, полученными при обычном расчете в цикле. Это подтверждает корректность реализации алгоритма. В данном запуске функция с применением ассемблерных вставок и технологии MMX показала меньшее время выполнения.

3 ВЫВОД

В ходе выполнения лабораторной работы была успешно освоена технология *MMX* для реализации параллельной обработки данных с использованием ассемблерных вставок в языке *C++*. Были изучены принципы работы с 64-разрядными регистрами, а также правила эффективной распаковки знаковых 8-битных чисел в 16-битные с сохранением знака с помощью инструкций сравнения и межсловной распаковки.

Реализованная программа корректно обрабатывает массивы из нескольких элементов с использованием системы команд *MMX*, а также демонстрирует преимущества технологии *MMX* в скорости выполнения арифметических операций над массивом данных.